



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

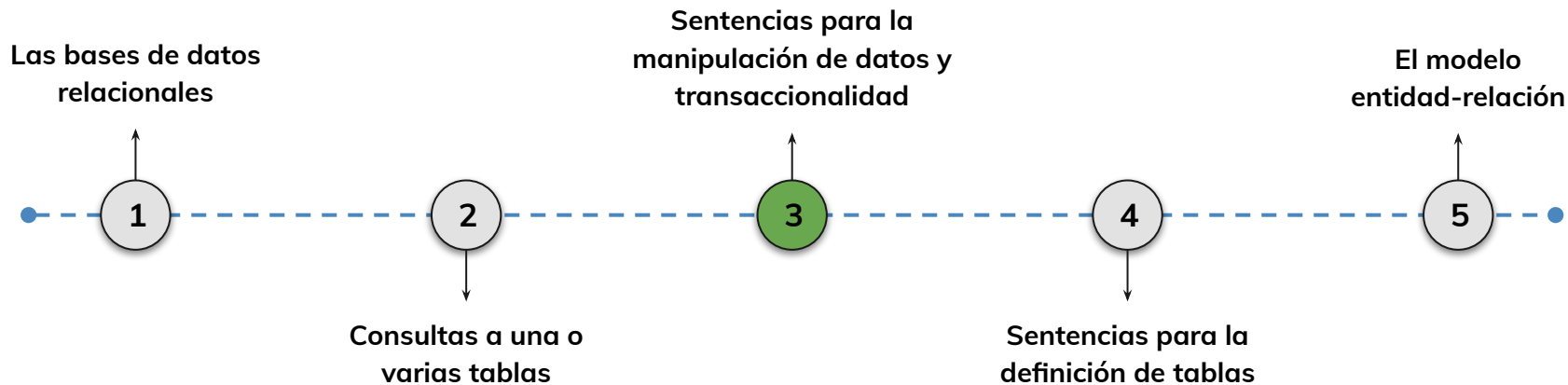


› Sentencias para la manipulación de datos y transaccionalidad - Parte 2

AE3: Utilizar lenguaje de manipulación de datos DML para la modificación de los datos existentes en una base de datos dando solución a un problema planteado del entrenamiento.

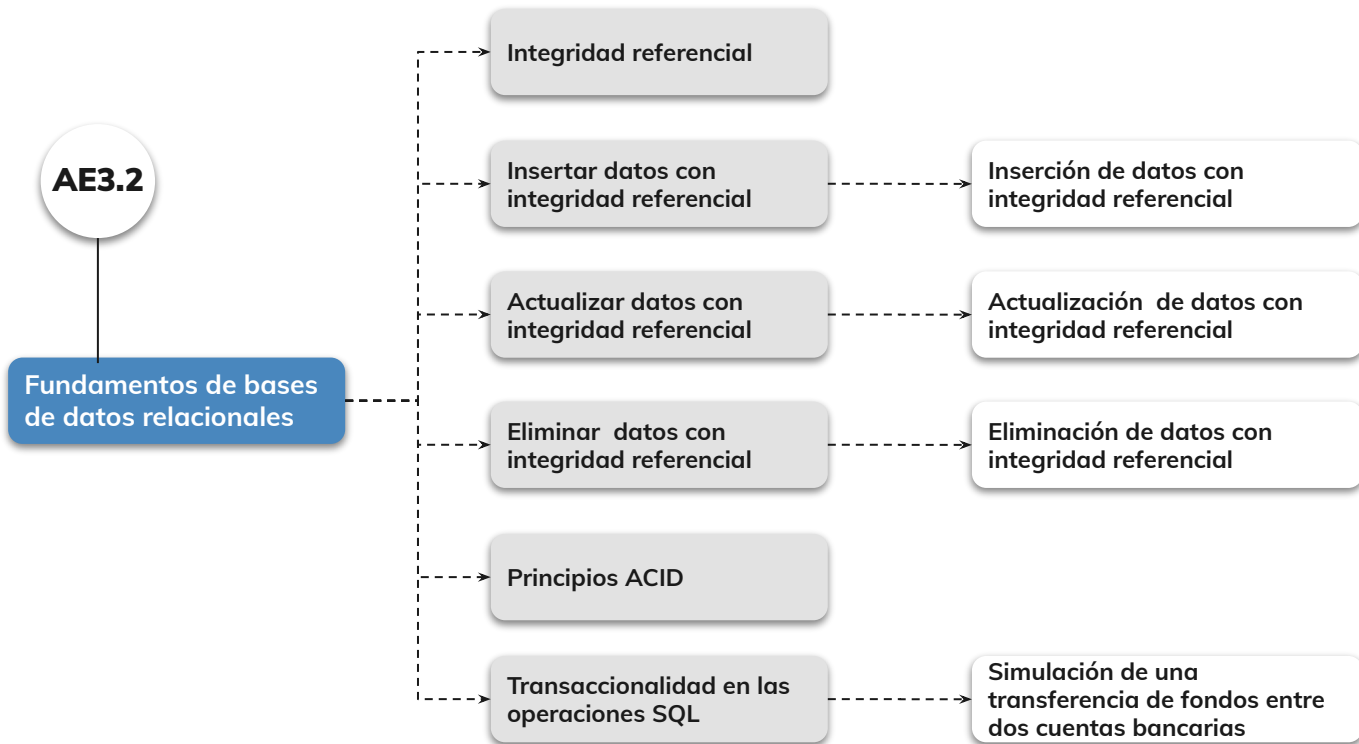
Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?



Objetivos de aprendizaje

¿Qué aprenderás?

- Conocer el concepto de integridad referencial
- Eliminar datos con integridad referencial
- Actualizar datos con integridad referencial
- Conocer los principios ACID
- Conocer qué es la transaccionalidad en las operaciones SQL
- Realizar transaccionalidades para confirmar y revertir cambios en la base de datos

Repaso clase anterior

¿Quedó alguna duda?

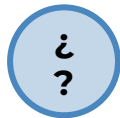
En la clase anterior trabajamos :

- Diferenciar los componentes principales de Data Manipulation Language (DML)
- Insertar datos en una tabla
- Crear ID autogenerados
- Aprendimos a actualizar información de una tabla utilizando el comando `UPDATE`
- Aprendimos a borrar información de una tabla utilizando el comando `DELETE`

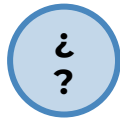
#Momentode Preguntas...



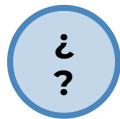
¿Qué es la integridad referencial?



¿Cómo actualizar datos con integridad referencial?



¿Cómo eliminar datos con integridad referencial?



¿Qué son las propiedades ACID?

Integridad referencial



Integridad referencial

¿Qué es la integridad referencial?

La integridad referencial en SQL es un concepto que garantiza que las relaciones entre tablas en una base de datos se mantengan consistentes y precisas. **Se utiliza para asegurar que las relaciones entre tablas se mantengan válidas, evitando inconsistencias y referencias a registros inexistentes.**

En esencia, la integridad referencial asegura que los valores en una columna (clave foránea) de una tabla coincidan con los valores en la columna (clave primaria) de otra tabla



Integridad referencial

Usos de la integridad referencial:

- 1.** Mantenimiento de la coherencia de datos: La integridad referencial garantiza que no se puedan crear relaciones incorrectas entre tablas, lo que asegura que los datos relacionados sean consistentes y precisos.
- 2.** Prevención de acciones no deseadas: Evita la eliminación o actualización de registros en tablas relacionadas que podrían afectar la coherencia de los datos.
- 3.** Mantener la estructura de la base de datos: La integridad referencial contribuye a la organización y estructura de la base de datos, al definir cómo las tablas se relacionan entre sí.

Integridad referencial

Implementación de la integridad referencial:

La integridad referencial se implementa a través de la definición de restricciones en las tablas que establecen cómo deben comportarse las relaciones.

Las restricciones más comunes son:

Clave Primaria (PRIMARY KEY): Define la columna o conjunto de columnas que actúan como clave primaria en una tabla. Cada valor en la clave primaria debe ser único

Clave Foránea (FOREIGN KEY): Define una columna que establece una relación con la clave primaria de otra tabla.



Integridad referencial



Ejemplo de implementación:

Supongamos que tienes dos tablas: "Clientes" y "Pedidos".

La tabla "Pedidos" tiene una clave foránea "id_cliente" que se relaciona con la clave primaria "id_cliente" en la tabla "Clientes".

Si intentas insertar un pedido con un "id_cliente" que no existe en la tabla "Clientes", la integridad referencial evitará esta acción.

Pedidos

id_orden	id_cliente
1	3

Clientes

id_cliente	nombre
3	Sofía

Insertar datos con integridad referencial



Insertar datos con integridad referencial



¿Cómo insertar datos con integridad referencial?:

Insertar datos con integridad referencial en SQL implica asegurarse de que los valores que ingreses en una columna con clave foránea sean válidos y correspondan a valores existentes en la columna de la tabla relacionada (clave primaria).

Contexto:

Supongamos que tienes dos tablas: "Clientes" y "Pedidos".

La tabla "Pedidos" tiene una clave foránea "id_cliente" que se relaciona con la clave primaria "id_cliente" en la tabla "Clientes".



Insertar datos con integridad referencial

Paso 1: Inserta un cliente en la tabla "Clientes" primero:

```
INSERT INTO Clientes (id_cliente, nombre)
VALUES (1, 'Cliente Ejemplo');
```

Paso 2: Insertar un pedido en la tabla "Pedidos" asegurándote de que la clave foránea "id_cliente" se refiera a un cliente existente en la tabla "Clientes":

```
INSERT INTO Pedidos (id_pedido, id_cliente, fecha_pedido)
VALUES (1, 1, '2023-08-17');
```

Insertar datos con integridad referencial

Analicemos el ejemplo anterior:

En este ejemplo, el valor 1 en la columna "id_cliente" de la tabla "Pedidos" corresponde al "id_cliente" insertado en la tabla "Clientes" en el paso anterior.

Esto mantiene la integridad referencial, ya que estás insertando un pedido relacionado con un cliente válido.

Si intentas insertar un pedido con un "id_cliente" que no existe en la tabla "Clientes", la base de datos arrojará un error de integridad referencial.

Es importante asegurarse de que los valores en las columnas con clave foránea sean válidos antes de realizar la inserción.

Siempre debes insertar primero los registros en la tabla principal (en este caso, "Clientes") y luego insertar registros en las tablas relacionadas (en este caso, "Pedidos") utilizando los valores correspondientes.

LIVE CODING

Ejemplo en vivo

Inserción de datos con Integridad referencial | Parte 1:

En este ejercicio, aprenderás a insertar datos en dos tablas relacionadas: **"Usuarios"** y **"Transacción"**.

- La tabla "Transacción" tiene una **clave foránea** "sender_user_id" que se relaciona con la **clave primaria** "user_id" en la tabla "Usuarios".
- La tabla "Transacción" tiene una **clave foránea** "receiver_user_id" que se relaciona con la **clave primaria** "user_id" en la tabla "Usuarios".

Aseguraremos la integridad referencial insertando primero usuarios y luego transacciones relacionadas con esos usuarios.

Tiempo: 15 minutos

LIVE CODING

Ejemplo en vivo

Inserción de datos con integridad referencial | Parte 2:

1. Insertar al menos tres transacciones en la tabla "Transacciones". Asegúrate de que cada transacción esté relacionada con uno de los usuarios que insertaste en el paso anterior.
2. Insertar al menos tres monedas en la tabla "Moneda".
3. Escribir una consulta para recuperar y mostrar todas las transacciones junto con el id del usuario al que pertenecen.

Tiempo: 15 minutos

Actualizar datos con integridad referencial



Actualizar datos con integridad referencial



¿Cómo actualizar datos con integridad referencial?:

Actualizar datos con integridad referencial en SQL involucra modificar registros de tal manera que se mantengan las relaciones y restricciones definidas entre tablas. Aquí tienes un ejemplo de cómo puedes hacerlo:

Contexto:

Supongamos que tienes dos tablas: "Autores" y "Libros". La tabla "Libros" tiene una clave foránea "id_autor" que se relaciona con la clave primaria "id_autor" en la tabla "Autores".

Actualizar datos con integridad referencial

Paso 1: Actualiza los datos del autor en la tabla "Autores":

```
UPDATE Autores
SET nombre_autor = 'Nuevo Nombre'
WHERE id_autor = 1;
```

Paso 2: actualiza los datos del libro relacionado con el autor actualizado:

```
UPDATE Libros
SET titulo_libro = 'Nuevo Título'
WHERE id_autor = 1;
```

Insertar datos con integridad referencial

Analizamos el ejemplo anterior:

En este ejemplo, primero actualizas el nombre de un autor en la tabla "Autores". Luego, actualizas el título de un libro en la tabla "Libros" relacionado con ese autor.

Asegúrate de que la restricción de clave foránea se mantenga al actualizar datos. La base de datos mantendrá la coherencia y evitará que actualices datos de tal manera que los registros relacionados sean inconsistentes.



LIVE CODING

Ejemplo en vivo

Actualización de datos con integridad referencial | Parte 1:

En este ejercicio, practicarás cómo actualizar datos en dos tablas relacionadas:
"Transacción" y "Usuarios".

La tabla "transacción" tiene una clave foránea "isender_user_id" que se relaciona con la clave primaria "user_id" en la tabla "Usuario".

Tiempo: 15 minutos

Actualización de Datos con Integridad Referencial | Parte 2:

1. Recuperar todos los cursos de la tabla "Usuarios" junto con el nombre del usuario al que pertenecen.
2. Actualizar el nombre de un usuario en la tabla "Usuario".
3. Escribir una consulta final para verificar que los datos se hayan actualizado correctamente y que la relación entre "Usuario" y "Transacción" se mantenga intacta.

Tiempo: 15 minutos

Eliminar datos con integridad referencial



Eliminar datos con integridad referencial

¿Cómo eliminar datos con integridad referencial?:

Eliminar datos con integridad referencial en SQL implica asegurarte de que las eliminaciones no rompan las relaciones y restricciones definidas entre las tablas.

Contexto:

Supongamos que tienes dos tablas: "Autores" y "Libros". La tabla "Libros" tiene una clave foránea "id_autor" que se relaciona con la clave primaria "id_autor" en la tabla "Autores".



Eliminar datos con integridad referencial



Paso 1: Eliminar un libro en la tabla "Libros" asegurándote de que la integridad referencial se mantenga:

```
DELETE FROM Libros  
WHERE id_libro = 1;
```

Paso 2: Eliminar al autor del libro que acabas de eliminar:

```
DELETE FROM Autores  
WHERE id_autor = 1;
```

Eliminar datos con integridad referencial

Analicemos el ejemplo anterior:

En este ejemplo, primero eliminas un libro de la tabla "Libros" y luego, eliminando al autor asociado en la tabla "Autores".

La base de datos mantendrá la coherencia y evitará eliminar registros que rompan relaciones.

Recuerda que al trabajar con integridad referencial, debes ser cuidadoso al eliminar registros en tablas relacionadas.



Eliminar datos con integridad referencial

¿Cómo eliminar datos en cascada?:

La opción "**CASCADE**" en las restricciones de clave foránea permite que las operaciones de eliminación o actualización en la tabla principal (la tabla referenciada por la clave foránea) se propaguen automáticamente a las tablas secundarias (las tablas que tienen la clave foránea).

Esto significa que puedes eliminar registros en la tabla principal y las filas relacionadas en las tablas secundarias también se eliminarán automáticamente.

Eliminar datos con integridad referencial

¿Cómo eliminar datos en cascada?:

Para eliminar datos en cascada simplemente debes implementar la opción "ON DELETE CASCADE" al crear la tabla. Aquí tienes un ejemplo

Paso 1:

```
CREATE TABLE Libros (  
  id_libro INT PRIMARY KEY,  
  titulo_libro VARCHAR(100),  
  id_autor INT,  
  FOREIGN KEY (id_autor)  
  REFERENCES Autores(id_autor) ON DELETE CASCADE  
);
```

Paso 2:

```
DELETE FROM Autores  
WHERE id_autor = 1;
```

En este ejemplo, cuando se elimina el autor con "id_autor" 1 de la tabla "Autores", el libro relacionado en la tabla "Libros" también se elimina automáticamente debido a la opción "ON DELETE CASCADE" en la restricción de clave foránea.

Inserción y eliminación de Datos con Integridad Referencial - Clientes y Pedidos

1. Crear dos tablas: "Clientes" y "Pedidos". Asegurarse de que la tabla Pedidos tenga al menos: `id_pedido`, `id_cliente` (Clave foránea que se relaciona con "id_cliente" en la tabla "Clientes" con `ON DELETE CASCADE`).
2. Eliminar uno de los clientes de la tabla "Clientes". Asegúrate de que los pedidos relacionados con ese cliente también se eliminen automáticamente debido a la restricción "ON DELETE CASCADE".

Tiempo: 15 minutos

LIVE CODING

Ejemplo en vivo

Eliminar de datos con integridad referencial

En este ejercicio, practicarás cómo eliminar datos en dos tablas relacionadas: "Transacción" y "Usuarios". La tabla "transacción" tiene una clave foránea "sender_user_id" que se relaciona con la clave primaria "user_id" en la tabla "Usuario".

Aseguraremos que la eliminación de un usuario también elimine automáticamente las transacciones relacionadas.

Tiempo: 15 minutos



Momento:

Time-out!



5 -10 min.



Principios ACID



Principios ACID



¿Qué son las propiedades ACID?:

Los principios ACID son un conjunto de propiedades que se aplican a las transacciones en bases de datos para garantizar la integridad y la coherencia de los datos en un entorno de procesamiento concurrente y en caso de fallos.

ACID es un acrónimo que representa las siguientes propiedades:

- Atomicidad (Atomicity)
- Consistencia (Consistency)
- Aislamiento (Isolation)
- Durabilidad (Durability)



Principios ACID

¿Qué significan las propiedades atomicidad y consistencia?

- **Atomicidad (Atomicity):** Una transacción se considera atómica si se trata como una unidad indivisible. Esto significa que todas las operaciones dentro de una transacción se ejecutan en su totalidad o no se ejecutan en absoluto. Si alguna operación dentro de una transacción falla, todas las operaciones anteriores se deshacen (rollback) y los cambios no se aplican.
- **Consistencia (Consistency):** Las transacciones deben llevar la base de datos desde un estado válido a otro estado válido. Esto garantiza que la base de datos siempre cumple con ciertas reglas y restricciones de integridad. Si una transacción no puede llevar a cabo una operación que mantiene la consistencia, se cancela.



Principios ACID



- **Aislamiento (Isolation):** Este principio asegura que las transacciones en ejecución sean independientes y no afecten entre sí, incluso cuando se ejecutan concurrentemente. El aislamiento evita que las transacciones en curso interfieren entre sí, lo que puede conducir a problemas como lecturas sucias, lecturas no repetibles, y escrituras fantasma.
- **Durabilidad (Durability):** Una vez que una transacción se completa exitosamente, sus cambios se mantienen permanentemente en la base de datos, incluso en caso de fallos posteriores del sistema. Esto garantiza que los datos modificados por una transacción persistan y no se pierdan.

Eliminar datos con integridad referencial

¿Para qué se utilizan estos principios?:

Los principios ACID son esenciales en aplicaciones que requieren un alto nivel de confiabilidad, consistencia y precisión en los datos, como sistemas de gestión de bases de datos (DBMS) utilizados en aplicaciones financieras, sistemas de reservas, sistemas de inventario y más

En resumen:

Los principios ACID son fundamentales para garantizar la integridad y la coherencia de los datos en aplicaciones críticas, especialmente en entornos donde la precisión y la confiabilidad son de suma importancia.

Transaccionalidad en las operaciones



Transaccionalidad en las operaciones

¿Qué es una transacción?

- La transaccionalidad en el contexto de las operaciones SQL se refiere a la capacidad de agrupar una serie de acciones o consultas en una unidad lógica llamada "transacción", que se ejecuta como una sola entidad indivisible.

¿Para que se utilizan?

- El propósito principal de las transacciones es garantizar la integridad de los datos, la coherencia y la consistencia en una base de datos, incluso en entornos de concurrencia y en presencia de fallos.



Transaccionalidad en las operaciones

Transaccionalidad y principios ACID:

- La transaccionalidad en las operaciones SQL se rige por las propiedades ACID (Atomicity, Consistency, Isolation y Durability)
- Esto permite ejecutar como una sola unidad operaciones complejas en las que intervienen varias sentencias de la base de datos, lo que proporciona fiabilidad y capacidad de recuperación ante fallos o errores.

Si una transacción falla, puede volver al estado anterior, lo que evita actualizaciones parciales o incoherentes de la base de datos

Transaccionalidad en las operaciones

Para poner en práctica la transaccionalidad en SQL, generalmente se siguen los siguientes pasos:

- 1. Inicio de la Transacción (BEGIN):** Se inicia una transacción utilizando la sentencia `START TRANSACTION(MySql)`.

A partir de este punto, todas las operaciones que se realicen formarán parte de la transacción actual.

2. Ejecución de Operaciones:

Durante la transacción, se ejecutan las operaciones de lectura y escritura en la base de datos como lo harías normalmente con sentencias SQL estándar (SELECT, INSERT, UPDATE, DELETE). Todas estas operaciones se consideran parte de la misma transacción hasta que se finalice.

Transaccionalidad en las operaciones

3. Confirmación (Commit): Una vez que todas las operaciones dentro de la transacción se han ejecutado exitosamente y no hay errores, se utiliza la sentencia COMMIT para confirmar la transacción. Esto significa que los cambios realizados en la base de datos durante la transacción se hacen permanentes y se guardan en la base de datos.

4. Cancelación (Rollback): Si en algún momento dentro de la transacción ocurre un error o se detecta alguna inconsistencia, se puede usar la sentencia ROLLBACK para cancelar todas las operaciones realizadas en la transacción. Esto devuelve la base de datos a su estado anterior a la transacción.



Transaccionalidad en las operaciones

En resumen:

Si una transacción se completa exitosamente, los cambios se confirman; si algo sale mal, se pueden revertir los cambios.

La implementación de transacciones es esencial para aplicaciones que requieren un control estricto sobre los cambios en la base de datos y la gestión de fallos.

LIVE CODING

Ejemplo en vivo

Simulación de una transferencia de fondos entre dos cuentas bancarias:

Deberás crear una transacción que garantice la integridad de los datos en caso de cualquier problema

1. Restar \$200.00 del saldo de la cuenta con ID 1.
2. Añadir \$200.00 al saldo de la cuenta con ID 2.
3. Confirmar la transacción.
4. Cancelar la transacción por saldo insuficiente

Tiempo: 15 minutos

LIVE CODING

Ejemplo en vivo

Supongamos que tienes una tabla llamada "Cuentas" con la siguiente estructura:

```
CREATE TABLE Cuentas (  
    ID INT PRIMARY KEY,  
    Saldo DECIMAL(10, 2)  
);  
  
INSERT INTO Cuentas (ID, Saldo)  
VALUES (1, 1000.00), (2, 1500.00);
```

¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ Conocer el concepto de integridad referencial
- ✓ Eliminar datos con integridad referencial
- ✓ Actualizar datos con integridad referencial
- ✓ Conocer los principios ACID
- ✓ Conocer qué es la transaccionalidad en las operaciones SQL
- ✓ Realizar transaccionalidades para confirmar y revertir cambios en la base de datos



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compártela o tráela al próximo encuentro sincrónico.



#Checkout

¿Qué les pareció la clase de hoy?



< **¡Muchas gracias!** >

